Midnight Signal ? Bug Fixes, Design Overhaul & Feature Roadmap

Executive summary

Midnight Signal now has a light-first, responsive browser studio alongside the existing terminal interface. The changes focus on local-first behavior, clearer controls, recoverable storage, safer agent feedback, document generation, and verified workspace edits. The project remains Python plus vanilla JavaScript and Ollama; no frontend framework was introduced.

Bugs found and fixed

1. **Fonts silently fell back to system defaults.** DM Sans, DM Mono, and Manrope now load from Google Fonts, with local system fallbacks when offline.

2. **Mode and model controls looked like browser-native selects.** Both are themed, keyboard-operable popovers. Mode options include icons, descriptions, and best-for guidance; models show size/capability and recommended-agent badges.

3. **Browser users could not tune model behavior.** A settings drawer now persists temperature, top-p, top-k, repeat penalty, context length, reply budget, theme, sound/volume, approval default, and sidebar preference. Removed the browser's hard-coded 8K request override; agent defaults remain appropriately bounded by the server.

4. **History depended on localStorage and could exceed its quota.** Conversations are backed up to IndexedDB; the app switches to IndexedDB when local history exceeds 4 MiB or localStorage fails. It provides restore/export feedback rather than silently discarding a failed save. Per-chat and all-chat Markdown exports are available, and deleting a conversation requires confirmation.

5. **Important failures were swallowed or poorly explained.** Logging and user-visible context were added to selected session, audio, doctor, server, and agent paths. Failures to reach Ollama, verify capabilities, or save browser history now produce clearer status or error messages.

6. **Agent approvals expired after a fixed two minutes.** The default is now ten minutes, configurable from 30 to 3,600 seconds, with a visible countdown and expired state.

7. **A final unmatched Markdown emphasis marker could disappear.** The terminal streaming renderer now flushes unmatched markers correctly at completion.

8. **The repository root contained unrelated generated files.** The five named samples were moved under `samples/`; packaging tests ensure local `.venv` and `.git` data are excluded from source archives.

9. **The dependency manifest was incomplete.** `requirements.txt` includes Ollama and the `python-docx` and ReportLab dependencies used by document generation.

10. **The browser had almost no command surface.** Ctrl/?+K opens a searchable command palette; `/` opens slash-command choices, and slash commands cover help, chat creation, export, settings, theme/mode changes, workspace selection, status, and search.

11. **Qwen2.5 tool-call recovery could accept malformed or incomplete calls.** Recovery remains narrowly validated against registered tools and argument schemas. Website-specific retries now require concrete tool edits and validation rather than accepting a prose-only success claim.

12. **Long replies and large histories could burden the UI.** Streaming Markdown updates are throttled to 70 ms; conversation rows are windowed above 100 matching chats, and filename search avoids scanning full attachment bodies.

13. **Ollama disconnects and unsupported agent models could leave unclear states.** The UI now surfaces connection/capability errors and the selected local model status.

14. **Phone-sized layouts were cramped.** The sidebar, header, composer, pickers, settings drawer, attachments, and approval flow have responsive behavior; a 390-pixel viewport was checked without horizontal page overflow.

15. **Agent tasks and files were difficult to inspect.** Tool activity is rendered as a status timeline; workspace switching is available from the header and remains scoped to the selected folder.

Design decisions: before and after

• **Before:** native mode/model selectors, system-font fallbacks, no browser settings, and an unfinished dark-first feel. **After:** a premium warm-white interface with a stronger type hierarchy, loaded display/body/monospace families, custom accessible pickers, and a light theme by default. Dark, high-contrast, auto, sunset, ember, cyber, matrix, ice, and noir variants are available.

• **Before:** basic composer and limited mobile affordances. **After:** responsive composer and attachment handling, drag/drop feedback, accessible shortcuts, clear disabled/streaming states, and a compact phone layout.

• **Before:** history failures were easy to miss and only localStorage was used. **After:** IndexedDB recovery, clear save/restore warnings, confirmation before deletion, and Markdown export reduce the chance of losing conversations.

• **Before:** agent actions and approvals lacked enough progress context. **After:** tool calls, results, approval countdowns, and failures are visible in the conversation timeline.

Features added or expanded

• Persistent settings and custom themes, plus optional synthesized interface sounds.

• Slash commands and a keyboard-searchable action/mode/recent-chat palette.

• 50 MiB total text-file attachment capacity per message (up to five files), checked by both browser and server.

• PDF and DOCX generation from Markdown through the workspace-scoped coding agent, with normal file-write approval.

• Markdown export for one conversation or the full local archive.

• Workspace selection from the header and opt-in LAN access for a phone on the same trusted Wi-Fi.

• More informative local Ollama/model status, model onboarding hints, and responsive approval feedback.

• Cleaner document output with a designed title, section styling, metadata, and page numbers.

How to use the improved browser UI

1. Install project dependencies and ensure Ollama is running with at least one model: `python midnight_signal.py --web` (Windows: `python midnight_signal.py --web`).

2. Use the top bar to choose a mode/model and workspace. Open **Settings** for sampling parameters, appearance, sound, and approval defaults.

3. Press Ctrl/?+K to search actions, modes, and recent conversations. Press `/` in an empty composer or type a slash command such as `/help`, `/theme dark`, `/mode agent`, or `/stats`.

4. To create files, select **Coding agent** and a model marked as supporting agent tools. Approve writes and commands in the visible confirmation cards. Ask for a PDF or DOCX with a workspace-relative output path.

5. To use a phone on the same trusted private Wi-Fi, start with `python midnight_signal.py --web --web-lan` (Windows: `python midnight_signal.py --web --web-lan`) and open the one-time private-network link printed by the laptop. Do not share that link or use it on public Wi-Fi; Windows Firewall may need to allow Python on the private network.

6. Export conversations before clearing browser data. Google Fonts are optional visual assets; system fallbacks preserve the interface when offline.

Verification and remaining limits

• Python regression suite: **80 tests passed, 1 skipped**. `node --check web\app.js` and `git diff --check` passed.

• Browser smoke checks verified persisted sampling/theme settings, the slash-command palette, accessible mode descriptions, responsive rendering at 390 px, and windowed navigation through 300 synthetic conversations.

• IndexedDB fallback was exercised with a 4.3 MB synthetic attachment history; the full attachment was backed up and restored after reload.

• A real local `huihui_ai/qwen2.5-coder-abliterate:7b` agent run created a 3,166-byte self-contained coffee-shop website in a temporary workspace. `tools.validate_website` returned no issues. This verifies one complete website-generation flow, not a guarantee for every open-ended model request.

• LAN authorization and API behavior are covered by automated tests, but a physical phone was not available for an end-to-end Wi-Fi test. LAN access is opt-in and requires both devices to share a trusted private network.

• The browser settings and history are local to that browser profile; use Markdown export for durable backups. Google-hosted font files require internet access, but the UI falls back to system fonts without them.